

LAPORAN PRAKTIKUM STRUKTUR DATA

QUEUE

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji Syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan materi Queue ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pemahaman penulis terhadap konsep Queue dalam struktur data serta implementasinya menggunakan bahasa pemrograman Java.

Dalam laporan ini, penulis membahas beberapa implementasi Queue, baik menggunakan array maupun struktur data bawaan seperti LinkedList, serta operasi – operasi dasar yang terdapat pada Queue.

Penulis menyadari bahwa lapoaran ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang. Akhir kata, semeoga laporan ini dapat memberikan manfaat bagi pembaca.

Padang, 27 April 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I.....	1
PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Tujuan Praktikum	1
1.3. Manfaat Praktikum.....	2
BAB II	2
PEMBAHASAN	3
2.1. Deskripsi Materi Queue	3
2.2. QueueArray_2511532005.....	3
2.3. QueueArrayDriver_2511532005.	5
2.4. ReverseData_2511532005.....	6
2.5. IterasiQueue_2511532005	7
2.6. QueueLinkedList_2511532005.....	8
2.7. Analisis Praktikum	9
BAB III.....	10
KESIMPULAN DAN SARAN.....	10
3.1 Kesimpulan.....	10
3.2 Saran.....	10
DAFTAR PUSTAKA	11

BAB I

PENDAHULUAN

1.1. Latar Belakang

Struktur data merupakan salah satu konsep penting dalam ilmu computer yang digunakan untuk mengelola dan menyimpan data secara efisien. Salah satu struktur data dasar yang sering digunakan adalah Queue (antrian).

Queue adalah struktur data yang menggunakan prinsip FIFO (First In First Out), yaitu data yang pertama kali masuk akan menjadi data pertama yang keluar. Konsep ini banyak digunakan dalam kehidupan sehari-hari seperti antrian di kasir, sistem antrian tiket, dan proses scheduling pada sistem operasi.

Dalam praktikum ini, dilakukan implementasi Queue menggunakan berbagai pendekatan, yaitu menggunakan array dan menggunakan struktur data bawaan Java seperti LinkedList. Selain itu, juga dipelajari bagaimana melakukan operasi seperti enqueue, dequeue, serta iterasi dan manipulasi data dalam Queue.

1.2. Tujuan Praktikum

1. Memahami konsep dasar Queue dalam struktur data.
2. Mengimplementasikan Queue menggunakan array.
3. Mengimplementasikan Queue menggunakan LinkedList.
4. Memahami operasi dasar Queue seperti enqueue, dequeue, peek, and isEmpty.
5. Mempelajari manipulasi data Queue seperti reverse dan itersi.

1.3. Manfaat Praktikum

1. Melatih logika terstruktur dalam mengelola alur data yang bersifat sekuensial dan bertumpuk.
2. Optimasi manajemen memori melalui perbandingan implementasi stack.
3. Kesiapan pemecahan masalah kompleks
4. Keahlian pemrograman modular
5. Efisiensi penulisan kode program

BAB II

PEMBAHASAN

2.1. Deskripsi Materi Queue

Queue adalah struktur data linear yang menggunakan prinsip FIFO (First In First Out). Elemen ditambahkan dari bagian belakang (rear) dan dihapus dari bagian depan (front).

Operasi dasar Queue meliputi:

1. Enqueue : menambahkan elemen ke belakang.
2. Dequeue : menghapus elemen dari depan.
3. Peek : melihat elemen terdepan.
4. isEmpty : mengecek apakah queue kosong.
5. isFull : mengecek apakah Queue penuh (untuk array).

Queue dapat diimplementasikan menggunakan array maupun LinkedList.

2.2. QueueArray_2511532005

Program ini merupakan implementasi Queue menggunakan array dengan konsep circular queue. Konsep ini digunakan agar array dapat dimanfaatkan secara efisien tanpa membuang ruang yang sudah tidak terpakai. Dalam implementasi ini, elemen ditambahkan dari belakang (rear) dan dihapus dari depan (front) sesuai prinsip FIFO.

```

1 package pekan4_2511532005;
2
3 public class QueueArray_2511532005 {
4     int front_2005, rear_2005, size_2005;
5     int capacity_2005;
6     int array_2005[];
7
8     public QueueArray_2511532005(int capacity_2005) {
9         this.capacity_2005 = capacity_2005;
10        front_2005 = this.size_2005 = 0;
11        rear_2005 = capacity_2005 - 1;
12        array_2005 = new int [this.capacity_2005];
13    }
14
15    boolean isFull(QueueArray_2511532005 queue) {
16        return (queue.size_2005 == queue.capacity_2005);
17    }
18
19    boolean isEmpty(QueueArray_2511532005 queue) {
20        return (queue.size_2005 == 0);
21    }
22
23    void enqueue_2005 (int item_2005) {
24        if (isFull(this))
25            return;
26        this.rear_2005 = (this.rear_2005 + 1) % this.capacity_2005;
27        this.array_2005[this.rear_2005] = item_2005;
28        this.size_2005 = this.size_2005 + 1;
29        System.out.println(item_2005 + " enqueued to queue");
30    }
31
32    int dequeue_2005() {
33        if (isEmpty(this))
34            return Integer.MIN_VALUE;
35        int item_2005 = this.array_2005[this.front_2005];
36        this.front_2005 = (this.front_2005 + 1) % this.capacity_2005;
37        this.size_2005 = this.size_2005 - 1;
38        return item_2005;
39    }
40
41    int front_2005() {
42        if (isEmpty(this))
43            return Integer.MIN_VALUE;
44        return this.array_2005[this.front_2005];
45    }
46    int rear_2005() {
47        if (isEmpty(this))
48            return Integer.MIN_VALUE;
49        return this.array_2005[this.rear_2005];
50    }
51    // mencetak elemen antrian
52    void display_2005() {
53        int i_2005;
54        if (front_2005 == rear_2005) {
55            System.out.printf("\nAntrian Kosong\n");
56            return;
57        }
58        // kunjungi dari belakang dan cetak
59        for (i_2005 = front_2005; i_2005 < rear_2005; i_2005++) {
60            System.out.printf(" %d <-- ", array_2005[i_2005]);
61        }
62        return;
63    }
64 }
65 }

```

Gambar 2.1. Kode program QueueArray_2511532005.

Kode dan Fungsi :

1. Variable front_2005, rear_2005, size_2005 : digunakan untuk mengatur posisi dan jumlah elemen dalam antrian.
2. Capacity_2005 : menyimpan kapasitas maksimum Queue
3. Array_2005 : digunakan sebagai tempat penyimpanan data.
4. Konstruktor berfungsi menginisialisasi Queue dengan mengatur ukuran, posisi awal front, rear, dan array.
5. Method isFull() digunakan untuk mengecek apakah Queue sudah penuh.

6. Method isEmpty() digunakan untuk mengecek apakah Queue.
7. Method enqueue_2005() berfungsi menambahkan elemen ke Queue dengan konsep circular (menggunakan modulo).
8. Method dequeue_2005() digunakan untuk menghapus elemen dari depan.
9. Method front_2005() dan rear_2005() digunakan untuk melihat elemen terdepan dan terakhir.
10. Method display_2005 digunakan untuk menampilkan isi Queue.

2.3. QueueArrayDriver_2511532005.

Kode program ini merupakan program utama yang digunakan untuk menguji implementasi QueueArray. Program ini berfungsi untuk memastikan bahwa semua operasi seperti enqueue, dequeue, dan penampilan data berjalan dengan benar.

```

1 package pekan4_2511532005;
2
3 public class QueueArrayDriver_2511532005 {
4     public static void main (String[] args) {
5         QueueArray_2511532005 queue = new QueueArray_2511532005(1000);
6         queue.enqueue_2005(10);
7         queue.enqueue_2005(20);
8         queue.enqueue_2005(30);
9         queue.enqueue_2005(40);
10        System.out.println("Item di depan" + queue.front_2005());
11        System.out.println("Item paling belakang " + queue.rear_2005());
12        System.out.println("Tampilan queue");
13        queue.display_2005();
14        System.out.println();
15        System.out.println(queue.dequeue_2005() + "dihapus dari queue");
16        System.out.println("Item di depan :" + queue.front_2005());
17        System.out.println("Item paling belakang :" + queue.rear_2005());
18        System.out.println("Tampilan queue setelah satu data dihapus");
19        queue.display_2005();
20    }
21 }
22

```

Gambar 2.2. Kode program QueueArrayDriver_2511532005.

Kode dan fungsi :

1. QueueArray queue = new QueueArray(1000); membuat objek queue dengan kapasitas 1000.

2. Queue.enqueue_2005(10); s.d. ...(40); Memanggil method enqueue_2005() untuk menambahkan beberapa elemen ke dalam Queue.
3. Menggunakan method front_2005() untuk menampilkan elemen terdepan.
4. Menggunakan method rear_2005() untuk menampilkan elemen terakhir.
5. Memanggil method display_2005() untuk mencetak isi Queue.
6. Menggunakan method dequeue_2005() untuk menghapus elemen dari Queue.
7. Menampilkan kembali kondisi Queue setelah dilakukan perubahan

2.4. ReverseData_2511532005

ReverseData_2511532005 merupakan program yang digunakan untuk membalik urutan elemen dalam Queue. Program ini memanfaatkan bantuan Stack untuk membalikkan urutan data karena sifat Stack yang LIFO (Last In First Out).

```

1 package pekan4_2511532005;
2 import java.util.LinkedList;
3 import java.util.Stack;
4 import java.util.Queue;
5
6 public class ReverseData_2511532005 {
7     public static void main (String[] args) {
8         Queue<Integer> q_2005 = new LinkedList<Integer>();
9         q_2005.add(1);
10        q_2005.add(2);
11        q_2005.add(3);
12        System.out.println( " sebelum reverse" + q_2005);
13        Stack<Integer> s_2005 = new Stack<Integer>();
14        while (!q_2005.isEmpty()) { // Q -> S
15            s_2005.push(q_2005.remove());
16        }
17        while (!s_2005.isEmpty()) { // S -> Q
18            q_2005.add(s_2005.pop());
19        }
20        System.out.println("Sesudah reverse =" + q_2005); // [3.2.1]
21
22    }
23
24 }
25

```

Gambar 2.3. Kode Program ReverseData_2511532005

Kode dan fungsi :

1. `Queue<Integer>q_2005 = new LinkedList<>();`

`Stack<Integer>s_2005 = new Stack<>();`

Kode ini membuat queue berbasis LinkedList dan membuat stack sebagai alat bantu.

2. `while (!s_2005.isEmpty()) {`
`s_2005.push(q_2005.remove()); }`

untuk memindahkan semua elemen dari queue ke stack.

3. `While(!q_2005.isEmpty()) {`
`q_2005.add(s_2005.pop()); }`

mengembalikan elemen ke queue dalam urutan terbalik.

2.5. IterasiQueue_2511532005

IterasiQueue_2511532005 merupakan program yang digunakan untuk menelusuri atau mengakses seluruh elemen dalam Queue. Proses iterasi dilakukan menggunakan iterator yang merupakan cara standar dalam Java untuk traversal struktur data.

```

1 package pekan4_2511532005;
2 import java.util.Iterator;
3 import java.util.Queue;
4 import java.util.LinkedList;
5
6 public class IterasiQueue_2511532005 {
7     public static void main(String args[]) {
8         Queue<String> q_2005 = new LinkedList<>();
9
10        q_2005.add("Praktikum");
11        q_2005.add("Struktur");
12        q_2005.add("Data");
13        q_2005.add("Dan");
14        q_2005.add("Algoritma");
15        Iterator<String> iterator = q_2005.iterator();
16        while (iterator.hasNext()) {
17            System.out.print(iterator.next() + " ");
18        }
19    }
20 }
21 }
22

```

Gambar 2.4. Kode program IterasiQueue_2511532005.

Kode dan fungsi :

1. `Iterator<String> iterator = q_2005.iterator();`
Membuat iterator untuk queue ini merupakan kode program inti.
2. `While (iterator.hasNext()) {`
`System.out.print(iterator.next() + “ “);`
`hasNext()` mengecek apakah masih ada data, sedangkan `next()` mengambil data satu per satu.

2.6. QueueLinkedList_2511532005

`QueueLinkedList_2511532005` merupakan implementasi Queue menggunakan struktur data `LinkedList` yang sudah tersedia dalam java Collections Framework. Pendekatan ini lebih sederhana karena tidak perlu mengatur indeks seperti pada array.

```

1 package pekan4_2511532005;
2 import java.util.*;
3
4 public class QueueLinkedList_2511532005 {
5     public static void main (String[] args) {
6
7         Queue<Integer> q_2005 = new LinkedList<>();
8         // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
9         for (int i_2005 = 0; i_2005 < 6; i_2005++)
10             q_2005.add(i_2005);
11
12         // menampilkan isi antrian
13         System.out.println("Elemen Antrian " + q_2005);
14
15         // untuk menghapus kepala antrian
16         int hapus_2005 = q_2005.remove();
17         System.out.println("Hapus elemen = " + hapus_2005);
18         System.out.println(q_2005);
19
20         // untuk melihat antrian terdepan
21         int depan_2005 = q_2005.peek();
22         System.out.println("Kepala antrian = " + depan_2005);
23
24         int banyak_2005 = q_2005.size();
25         System.out.println("Size Antrian =" + banyak_2005);
26     }
27
28 }

```

Gambar 2.5. Kode program `QueueLinkedList_2511532005`.

Kode dan fungsi :

1. `Queue<Integer> q_2005 = new LinkedList<>();` = Membuat queue
2. `for (int i_2005 = 0; i_2005 < 6; i_2005++)`

`q_2005.add(i_2005);`

berfungsi untuk menambahkan elemen 0-5 ke queue.

3. `int hapus_2005 = q_2005.remove();` untuk menghapus elemen depan
4. `int depan_2005 = q_2005.peek();` untuk melihat elemen depan tanpa menghapus.
5. `int banyak_2005 = q_2005.size();` untuk menghitung jumlah elemen

2.7. Analisis Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat dianalisis bahwa implementasi struktur data Queue memiliki beberapa pendekatan berbeda namun tetap menggunakan prinsip FIFO (First In First Out), dimana pada QueueArray diperlukan pengelolaan indeks seperti front dan rear serta penerapan konsep circular queue agar penggunaan memori lebih efisien meskipun implementasinya lebih kompleks, sedangkan pada QueueLinkedList penggunaan struktur bawaan Java membuat implementasi menjadi lebih sederhana dan fleksibel tanpa perlu mengatur indeks secara manual, selain itu praktikum juga menunjukkan bahwa Queue dapat dikombinasikan dengan struktur data lain seperti Stack pada program ReverseData untuk membalikan urutan data, serta penggunaan iterator pada IterasiQueue memungkinkan proses traversal data dilakukan secara aman tanpa mengubah isi Queue, sehingga secara keseluruhan dapat disimpulkan bahwa pemilihan metode implementasi Queue harus disesuaikan dengan kebutuhan program baik dari segi efisiensi, kemudahan, maupun fleksibilitas.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa Queue merupakan struktur data yang menggunakan prinsip FIFO. Queue dapat diimplementasikan menggunakan array maupun LinkedList. Implementasi menggunakan array memerlukan pengelolaan indeks secara manual, sedangkan LinkedList lebih sederhana karena memanfaatkan library yang sudah tersedia. Selain itu, operasi tambahan seperti reverse dan iterasi dapat dilakukan untuk manipulasi data dalam Queue.

3.2 Saran

Dalam mempelajari struktur data, disarankan untuk :

1. lebih sering berlatih implementasi berbagai struktur data.
2. Memahami konsep dasar sebelum implementasi.
3. Mencoba berbagai variasi kasus penggunaan Queue.
4. Mengembangkan program yang lebih kompleks sebagai latihan lanjutan.

DAFTAR PUSTAKA

- Kartik. (2025). *Queue - Linked List Implementation*. Retrieved from geeksforgeeks:
<https://www.geeksforgeeks.org/dsa/queue-linked-list-implementation/>
- Maulana, R. (2023). *Struktur Data Queue: Pengertian, Fungsi, dan Jenisnya*.
Retrieved from dicoding.
- rajput. (2025). *Queue using Array - Simple Implementation*. Retrieved from
geeksforgeeks.

TUGAS PRAKTIKUM 3 StackArray

1. Perintah Tugas

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan **struktur data Queue (Antrian)** menggunakan **array dalam bahasa Java**.

Studi kasus yang digunakan adalah **antrian pelanggan di loket pelayanan**. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip **FIFO (First In First Out)**.

Program harus mampu:

- Menambahkan pelanggan ke antrian (**enqueue**)
- Menghapus pelanggan dari antrian (**dequeue**)
- Menampilkan isi antrian
- Mengecek kondisi antrian (kosong atau penuh)

Gambar 1.1 Perintah Tugas Praktikum Pekan 4.

2. Kode Program AntrianLoket_2511532005

Berikut Kode Program AntrianLoket_2511532005.

```

1 package pekan4_2511532005;
2
3 public class AntrianLoket_2511532005 {
4     int front_2005, rear_2005, size_2005;
5     int capacity_2005;
6     String antrian_2005[];
7
8     public AntrianLoket_2511532005(int capacity_2005) {
9         this.capacity_2005 = capacity_2005;
10        front_2005 = this.size_2005 = 0;
11        rear_2005 = capacity_2005 - 1;
12        antrian_2005 = new String[this.capacity_2005];
13    }
14
15    boolean isFull_2005(AntrianLoket_2511532005 antrian) {
16        return (antrian.size_2005 == antrian.capacity_2005);
17    }
18
19    boolean isEmpty_2005(AntrianLoket_2511532005 antrian) {
20        return (antrian.size_2005 == 0);
21    }
22
23    void enqueue_2005(String namaPelanggan_2005) {
24        if (isFull_2005(this))
25            return;
26        this.rear_2005 = (this.rear_2005 + 1) % this.capacity_2005;
27        this.antrian_2005[this.rear_2005] = namaPelanggan_2005;
28        this.size_2005 = this.size_2005 + 1;
29    }
30
31    String dequeue_2005() {
32        if (isEmpty_2005(this))
33            return null;
34        String namaPelanggan_2005 = this.antrian_2005[this.front_2005];
35        this.front_2005 = (this.front_2005 + 1) % this.capacity_2005;
36        this.size_2005 = this.size_2005 - 1;
37        return namaPelanggan_2005;
38    }
39
40    void display_2005() {
41        if (front_2005 == rear_2005 && size_2005 == 0) {
42            System.out.println("Antrian kosong");
43            return;
44        }
45        System.out.println("Isi antrian:");
46        int i_2005 = front_2005;
47        for (int nomor_2005 = 1; nomor_2005 <= size_2005; nomor_2005++) {
48            System.out.println(nomor_2005 + ". " + antrian_2005[i_2005]);
49            i_2005 = (i_2005 + 1) % capacity_2005;
50        }
51    }
52
53    void reverse_2005() {
54        if (size_2005 <= 1) return;
55
56        String[] temp_2005 = new String[capacity_2005];
57        int j_2005 = 0;
58        int index_2005 = front_2005;
59        for (int i_2005 = 0; i_2005 < size_2005; i_2005++) {
60            temp_2005[j_2005++] = antrian_2005[index_2005];
61            index_2005 = (index_2005 + 1) % capacity_2005;
62        }
63
64        front_2005 = 0;
65        rear_2005 = size_2005 - 1;
66        for (int i_2005 = 0; i_2005 < size_2005; i_2005++) {
67            antrian_2005[i_2005] = temp_2005[size_2005 - 1 - i_2005];
68        }
69    }
70 }
71

```

Gambar 2.1 Kode Program AntrianLoket_2511532005.

Penjelasan fungsi atau kegunaan dari kode program ini :

a. Deklarasi Class

int front_2005, rear_2005, size_2005, capacity_2005; String
antrian_2005[];

b. Constructor

```

public AntrianLoket_2511532005(int capacity_2005) {
    this.capacity_2005 = capacity_2005;
    front_2005 = this.size_2005 = 0;
    rear_2005 = capacity_2005 - 1;
    antrian_2005 = new String[this.capacity_2005]; }

```

ini berfungsi untuk menginisialisasi ukuran antrian. Dimana antrian dimulai dari indeks 0. Serta ini juga berfungsi untuk membuat array sebagai menyimpan data.

c. Method isFull

```

boolean isFull_2005(AntrianLoket_2511532005 antrian) {
    return (antrian.size_2005 == antrian.capacity_2005);
}

```

Fungsi nya untuk mengecek apa antrian sudah penuh lalu mengembalikan true jika penuh.

d. Method isEmpty

```

boolean isEmpty_2005(AntrianLoket_2511532005 antrian) {
    return (antrian.size_2005 == 0);
}

```

Fungsinya untuk mengecek apakah antrian kosong dan mengembalikan true jika kosong.

e. Method enqueue

```

void enqueue_2005(String namaPelanggan_2005) {
    if (isFull_2005(this))
        return;
    this.rear_2005 = (this.rear_2005 + 1 ) % this.capacity_2005;
    this.antrian_2005[this.rear_2005] = namaPelanggan_2005;
    this.size_2005 = this.size_2005 + 1;
}

```

Fungsi ini untuk menambahkan pelanggan ke antrian.

f. Method dequeue

```
String dequeue_2005() {
    if (isEmpty_2005(this))
        return null;
    String          namaPelanggan_2005          =
this.antrian_2005[this.front_2005];
    this.front_2005 = (this.front_2005 + 1) % this.capacity_2005;
    this.size_2005 = this.size_2005 - 1;
    return namaPelanggan_2005;
}
```

Fungsinya untuk menghapus pelanggan paling depan.

g. Method display

```
void display_2005() {
    if (front_2005 == rear_2005 && size_2005 == 0) {
        System.out.println("Antrian kosong");
        return;
    }
    System.out.println("Isi antrian:");
    int i_2005 = front_2005;
    for (int nomor_2005 = 1; nomor_2005 <= size_2005;
nomor_2005++) {
        System.out.println(nomor_2005 + ". " +
antrian_2005[i_2005]);
        i_2005 = (i_2005 + 1) % capacity_2005;
    }
}
```

Fungsi untuk menampilkan semua isi antrian.

h. Method reverse

```
void reverse_2005() {
    if (size_2005 <= 1) return;
```

```

String[] temp_2005 = new String[capacity_2005];
int j_2005 = 0;
int index_2005 = front_2005;
for (int i_2005 = 0; i_2005 < size_2005; i_2005++) {
    temp_2005[j_2005++] = antrian_2005[index_2005];
    index_2005 = (index_2005 + 1) % capacity_2005;
}

front_2005 = 0;
rear_2005 = size_2005 - 1;
for (int i_2005 = 0; i_2005 < size_2005; i_2005++) {
    antrian_2005[i_2005] = temp_2005[size_2005 - 1 - i_2005];
}
}
}

```

Fungsi nya untuk membalik urutan antrian.

3. Kode Program AntrianLoketDriver_2511532005

```

1 package pekan4_2511532005;
2 import java.util.Scanner;
3
4 public class AntrianLoketDriver_2511532005 {
5     public static void main(String[] args) {
6         Scanner scanner_2005 = new Scanner(System.in);
7         AntrianLoket_2511532005 antrian_2005 = new AntrianLoket_2511532005(100);
8
9         while (true) {
10             System.out.println("\n==PROGRAM ANTRIAN LOKET==");
11             System.out.println("1. Tambah Antrian");
12             System.out.println("2. Hapus Antrian");
13             System.out.println("3. Tampilkan Antrian");
14             System.out.println("4. Reverse");
15             System.out.println("5. Keluar");
16             System.out.print("Pilih menu: ");
17
18             int pilihan_2005 = scanner_2005.nextInt();
19             scanner_2005.nextLine();
20
21             switch (pilihan_2005) {
22                 case 1:
23                     System.out.print("Masukkan nama pelanggan: ");
24                     String nama_2005 = scanner_2005.nextLine();
25                     antrian_2005.enqueue_2005(nama_2005);
26                     System.out.println("Data berhasil ditambahkan ke antrian");
27                     break;
28
29                 case 2:
30                     String pelanggan_2005 = antrian_2005.dequeue_2005();
31                     if (pelanggan_2005 != null) {
32                         System.out.println(pelanggan_2005 + " telah dilayani");
33                     } else {
34                         System.out.println("Antrian kosong!");
35                     }
36                     break;

```

```

37
38 case 3:
39     antrian_2005.display_2005();
40     break;
41
42 case 4:
43     antrian_2005.reverse_2005();
44     System.out.println("Antrian berhasil di-reverse");
45     antrian_2005.display_2005();
46     break;
47
48 case 5:
49     System.out.println("Terima kasih!");
50     return;
51
52 default:
53     System.out.println("Pilihan tidak valid!");
54 }
55 }
56 }
57
58

```

Gambar 3.1 Kode Program AntrianLoketDriver_2511532005.

Penjelasan fungsi atau kegunaan dari kode program ini :

a. Import dan Main Class

```
import java.util.Scanner;
```

```
public class AntrianLoketDriver_2511532005 {
    public static void main(String[] args) {
```

Fungsi nya untuk tempat program dijalankan.

b. Inisialisasi Scanner dan Queue

```
Scanner scanner_2005 = new Scanner(System.in);
```

```
AntrianLoket_2511532005 antrian_2005 = new
AntrianLoket_2511532005(100);
```

Fungsinya untuk membaca input user dan membuat objek antrian dengan kapasitas 100.

c. Perulangan Menu

```
while (true) {
```

fungsi nya membuat program terus berjalan sampai user memilih keluar, dan loop tak terbatas.

d. Tampilan menu

```
System.out.println("\n===PROGRAM ANTRIAN LOKET===");
```

```
System.out.println("1. Tambah Antrian");
```

```
System.out.println("2. Hapus Antrian");
```

```
System.out.println("3. Tampilkan Antrian");
```

```
System.out.println("4. Reverse");
```

```
System.out.println("5. Keluar");
```

```
System.out.print("Pilih menu: ");
```

Fungsinya untuk menampilkan pilihan menu kepada user.

e. Input Pilihan User

```
int pilihan_2005 = scanner_2005.nextInt();
```

```
scanner_2005.nextLine();
```

funksinya untuk membaca angka pilihan dan membersihkan buffer input.

f. Percabangan switch, berfungsi untuk menentukan aksi berdasarkan pilihan user.

g. Case 1 untuk menambah antrian

h. Case 2 untuk menghapus antrian

i. Case 3 untuk menampilkan antrian yang sudah di inputkan.

j. Case 4 untuk membalikkan antrian

k. Case 5 untuk keluar dari program.terakhir ada default untuk menangani input yang tidak sesuai menu.

4. Output Dari Kode Program

```
===PROGRAM ANTRIAN LOKET===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: ica  
Data berhasil ditambahkan ke antrian
```

```
Pilih menu: 1  
Masukkan nama pelanggan: sasuke  
Data berhasil ditambahkan ke antrian
```

```
Pilih menu: 3
Isi antrian:
1. ica
2. sasuke
```

```
Pilih menu: 4
Antrian berhasil di-reverse
Isi antrian:
1. sasuke
2. ica
```

```
Pilih menu: 2
sasuke telah dilayani
```

```
Pilih menu: 2
ica telah dilayani
```

```
Pilih menu: 5
Terima kasih!
```